

INTRO TO SCIENTIFIC COMPUTING IN PYTHON

LECTURE 2

Zan Ahmad

Department of Applied Mathematics and Statistics,
Johns Hopkins University

January 9, 2026

Part I

PRACTICE PROBLEMS

TWO SUM — PROBLEM STATEMENT

Given an array of integers `nums` and an integer `target`, return the indices `i` and `j` such that:

$$\text{nums}[i] + \text{nums}[j] = \text{target}, \quad i \neq j$$

You may assume:

- ▶ exactly one valid solution exists,
- ▶ the answer should return the smaller index first.

Goal: design an algorithm that runs in $O(n)$ **time** and $O(n)$ **space**.

EXAMPLES

Example 1

- ▶ Input: `nums = [3, 4, 5, 6], target = 7`
- ▶ Output: `[0, 1]`

Example 2

- ▶ Input: `nums = [4, 5, 6], target = 10`
- ▶ Output: `[0, 2]`

Example 3

- ▶ Input: `nums = [5, 5], target = 10`
- ▶ Output: `[0, 1]`

NAIVE APPROACH

BRUTE FORCE SOLUTION

```
def twoSum(nums, target):  
    n = len(nums)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if nums[i] + nums[j] == target:  
                return [i, j]
```

Why this works:

- ▶ checks every possible pair,
- ▶ guaranteed to find the unique solution.

Complexity:

- ▶ Time: $O(n^2)$ — nested loops over the array
- ▶ Space: $O(1)$ — no extra data structures

HOW CAN WE DO BETTER?

ONE-PASS HASH MAP SOLUTION

```
def twoSum(nums, target):  
    seen = {}  
    for i, num in enumerate(nums):  
        complement = target - num  
        if complement in seen:  
            return [seen[complement], i]  
        seen[num] = i
```

Idea:

- ▶ store numbers we have already seen,
- ▶ check if the needed complement exists,
- ▶ solve the problem in a single pass.

Complexity:

- ▶ Time: $O(n)$ — each element processed once
- ▶ Space: $O(n)$ — dictionary stores up to n elements

WHY THE HASH MAP SOLUTION IS OPTIMAL

- ▶ Eliminates redundant comparisons
- ▶ Trades memory for speed
- ▶ Leverages $O(1)$ average dictionary lookup

Key insight:

Good algorithms remember what they have already learned.

CONTAINS DUPLICATE — PROBLEM STATEMENT

Given an integer array `nums`, return `True` if any value appears **at least twice** in the array, and return `False` if every element is distinct.

Goal: design an algorithm that runs in $O(n)$ **time** (and uses reasonable memory).

Key constraint: We only need to know whether a duplicate exists (not where it occurs).

EXAMPLES

Example 1

- ▶ Input: `nums = [1, 2, 3, 1]`
- ▶ Output: `True`

Example 2

- ▶ Input: `nums = [1, 2, 3, 4]`
- ▶ Output: `False`

Example 3

- ▶ Input: `nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2]`
- ▶ Output: `True`

NAIVE SOLUTION

BRUTE FORCE SOLUTION

```
def containsDuplicate(nums):  
    n = len(nums)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if nums[i] == nums[j]:  
                return True  
    return False
```

Why this works:

- ▶ checks every pair of indices (i, j) ,
- ▶ returns immediately when a match is found.

Complexity:

- ▶ Time: $O(n^2)$ — nested loops compare many pairs
- ▶ Space: $O(1)$ — no extra data structures

HOW CAN WE DO BETTER?

ONE-PASS SET SOLUTION

```
def containsDuplicate(nums):  
    seen = set()  
    for x in nums:  
        if x in seen:  
            return True  
        seen.add(x)  
    return False
```

Idea:

- ▶ maintain a set of values we have already seen,
- ▶ when you encounter a value already in the set, it is a duplicate.

Complexity:

- ▶ Time: $O(n)$ — one pass, with $O(1)$ average membership checks
- ▶ Space: $O(n)$ — in the worst case, the set stores all elements

WHY THE SET SOLUTION IS OPTIMAL

Key observation: Detecting duplicates is equivalent to asking:

“Have I seen this value before?”

A set is designed for exactly this:

- ▶ membership test ($x \text{ in seen}$) is $O(1)$ on average,
- ▶ duplicates are naturally prevented by the set structure.

Key takeaway:

Use a set when the main operation is membership testing and uniqueness.

VALID ANAGRAM — PROBLEM STATEMENT

Given two strings `s` and `t`, return `True` if `t` is an anagram of `s`, and `False` otherwise.

An **anagram** means:

- ▶ both strings contain the same characters,
- ▶ with the same frequencies,
- ▶ possibly in a different order.

Goal: solve efficiently in $O(n)$ **time**.

EXAMPLES

Example 1

- ▶ Input: `s = "anagram", t = "nagaram"`
- ▶ Output: `True`

Example 2

- ▶ Input: `s = "rat", t = "car"`
- ▶ Output: `False`

Example 3

- ▶ Input: `s = "aacc", t = "ccac"`
- ▶ Output: `False`

SCRATCH WORK FOR INITIAL SOLUTION IDEA

BASELINE SOLUTION: SORTING

```
def isAnagram(s, t):  
    if len(s) != len(t):  
        return False  
    return sorted(s) == sorted(t)
```

Why this works:

- ▶ sorting puts characters in a canonical order,
- ▶ anagrams become identical after sorting.

Complexity:

- ▶ Time: $O(n \log n)$ — sorting dominates
- ▶ Space: $O(n)$ — sorting typically allocates new arrays

HOW CAN WE DO BETTER?

VALID ANAGRAM: TWO DICTIONARY COUNTING

```
def isAnagram(s, t):  
    if len(s) != len(t):  
        return False  
  
    count_s = {}  
    count_t = {}  
  
    for ch in s:  
        count_s[ch] = count_s.get(ch, 0) + 1  
  
    for ch in t:  
        count_t[ch] = count_t.get(ch, 0) + 1  
  
    return count_s == count_t
```

Complexity:

- ▶ Time: $O(n)$ — two linear passes
- ▶ Space: $O(1)$ for fixed alphabet (or $O(k)$ for k unique characters)

WHY THE DICTIONARY COUNTING METHOD WORKS

Core idea: Anagrams have identical frequency counts.

This solution enforces that by:

- ▶ incrementing counts for each character in *s* and *t*,
- ▶ compares the counts for both strings.

At the end:

- ▶ if the frequency counts are identical, the strings match exactly, return `True`,
- ▶ if not, we know they are not anagrams, return `False`.

Key takeaway:

Sorting compares order; counting compares structure.
Counting is the more direct strategy here.

Part II

NUMPY: SCIENTIFIC COMPUTING AT SCALE

WHY WE NEED NUMPY FOR SCIENTIFIC PROGRAMMING

In scientific programming, we often work with **vectors**, **matrices**, and **grids of data** that can contain millions of numbers.

Pure Python can represent these objects using lists-of-lists, but that approach has two major limitations:

1. It is **slow at scale**, because Python loops execute one element at a time with significant overhead.
2. It is **awkward for mathematics**, because lists do not naturally support linear algebra operations.

NumPy solves both problems by providing an efficient array object, implemented in optimized compiled code, with a clean mathematical interface.

WHAT NUMPY GIVES YOU

NumPy provides an object called an **ndarray** (“N-dimensional array”).

An ndarray is designed for scientific work:

- ▶ it stores numerical data in a **contiguous block of memory** (like C/Fortran),
- ▶ it supports fast operations on entire arrays (“**vectorization**”),
- ▶ it connects directly to scientific libraries (SciPy, scikit-learn, etc.).

Mental model: Python lists are general containers. NumPy arrays are specialized numerical data structures.

MOTIVATION: MATRIX MULTIPLICATION

Matrix multiplication is a basic building block of scientific computing:

- ▶ linear systems ($Ax = b$),
- ▶ least squares regression,
- ▶ eigenvalue problems,
- ▶ neural networks and machine learning.

Before using NumPy, it is important to see what this looks like in pure Python and why that becomes a bottleneck.

MATRIX MULTIPLICATION IN PURE PYTHON (LISTS)

Suppose A is $m \times n$ and B is $n \times p$. The product $C = AB$ is $m \times p$.

A direct implementation using lists-of-lists:

```
def matmul_python(A, B):
    m = len(A)
    n = len(A[0])
    p = len(B[0])

    # initialize C with zeros
    C = [[0.0 for _ in range(p)] for _ in range(m)]

    for i in range(m):
        for j in range(p):
            total = 0.0
            for k in range(n):
                total += A[i][k] * B[k][j]
            C[i][j] = total

    return C
```

This is correct, but it uses **three nested Python loops**.

WHY THE PURE PYTHON VERSION IS SLOW

The triple loop performs mnp multiply-add operations. That is unavoidable.

However, the real issue is **where the loop runs**:

- ▶ In the pure Python version, the loops run in the Python interpreter.
- ▶ Each loop iteration has interpreter overhead (type checks, object handling, etc.).

In scientific computing, the goal is to move “inner loops” out of Python and into optimized compiled code.

NumPy does exactly that.

MATRIX MULTIPLICATION WITH NUMPY

With NumPy, matrix multiplication becomes a single high-level operation:

```
import numpy as np

C = A @ B          # preferred syntax
# or
C = np.matmul(A, B) # explicit function call
```

This is not “magic.” NumPy delegates the heavy computation to optimized compiled routines (often BLAS/LAPACK libraries).

Key idea: the mathematical operation is the same, but the low-level implementation is vastly faster.

VECTORIZATION: THE CORE NUMPY IDEA

Vectorization means applying operations to entire arrays at once, instead of looping in Python.

Example: if x is a vector, then computing $y_i = x_i^2$ for all i is:

- ▶ in Python: a loop over i ,
- ▶ in NumPy: one expression $y = x**2$.

Even though the computation still happens element-by-element internally, NumPy executes it in compiled code, which avoids Python loop overhead and is much faster.

CREATING ARRAYS

NumPy arrays are usually created from Python lists or from special constructors:

```
import numpy as np

x = np.array([1, 2, 3])           # 1D array (vector)
A = np.array([[1, 2], [3, 4]])    # 2D array (matrix)

z = np.zeros((3, 4))              # 3x4 array of zeros
o = np.ones((2, 5))               # 2x5 array of ones
I = np.eye(4)                     # 4x4 identity matrix
```

Key idea: shape is written as a tuple, like (rows, cols).

ARRAY PROPERTIES: SHAPE, DTYPE, NDIM

Every ndarray has key attributes:

```
A.shape    # tuple of dimensions, e.g. (3, 4)
A.ndim     # number of dimensions, e.g. 2
A.dtype    # data type, e.g. float64, int64
A.size     # total number of entries, e.g. 12
```

Why dtype matters: NumPy arrays store uniform numeric types for efficiency. A Python list can hold mixed types; a NumPy array typically does not.

RANGES: ARANGE AND Linspace

Two common ways to generate 1D grids:

```
x = np.arange(0, 10, 2)      # 0, 2, 4, 6, 8   (step-based)
t = np.linspace(0, 1, 5)     # 0.0, 0.25, 0.5, 0.75, 1.0 (count-based)
```

Use `arange` when you want a fixed step size.

Use `linspace` when you want a fixed number of sample points (common in numerical methods).

INDEXING AND SLICING

Indexing looks similar to Python lists, but works in multiple dimensions:

```
A = np.array([[10, 20, 30],  
              [40, 50, 60]])
```

```
A[0, 1]      # 20  
A[1, :]      # entire second row  
A[:, 2]      # entire third column  
A[:, 0:2]    # first two columns
```

Key idea: the comma separates dimensions.

BOOLEAN MASKING (FILTERING IN NUMPY)

Boolean masks let you select entries that satisfy a condition:

```
x = np.array([1, 4, 2, 9, 3])

mask = x > 3          # array([False,  True, False,  True, False])
x[mask]              # array([4, 9])

# shorthand
x[x > 3]             # array([4, 9])
```

This corresponds exactly to the “filtering” pattern from earlier, but expressed in a clean vectorized form.

ELEMENTWISE OPERATIONS

NumPy operations are typically elementwise:

```
x = np.array([1.0, 2.0, 3.0])
```

```
x + 5      # array([6., 7., 8.])
```

```
x**2      # array([1., 4., 9.])
```

```
np.sin(x)  # applies sin to every element
```

Key idea: You write the math as if it were scalar, and NumPy applies it entry-by-entry.

BROADCASTING: HOW SHAPES CAN STILL WORK TOGETHER

Broadcasting is NumPy's rule system for combining arrays of different shapes.

Example: add a scalar to a vector:

```
x = np.array([1, 2, 3])  
x + 10          # array([11, 12, 13])
```

Example: subtract a vector from each row of a matrix:

```
A = np.array([[1, 2, 3],  
              [4, 5, 6]])  
v = np.array([10, 20, 30])
```

$A - v$

Broadcasting avoids explicit loops and is one reason NumPy code stays short and fast.

REDUCTIONS: SUM, MEAN, MAX

Reductions collapse an array into a smaller object by aggregating values:

```
x = np.array([1, 2, 3, 4])
```

```
np.sum(x)      # 10
```

```
np.mean(x)     # 2.5
```

```
np.max(x)      # 4
```

For 2D arrays, you can reduce along an axis:

```
A = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

```
np.sum(A, axis=0)  # column sums: [5, 7, 9]
```

```
np.sum(A, axis=1)  # row sums:    [6, 15]
```

RESHAPING AND TRANSPOSE

Reshaping changes the view of the data without changing entries:

```
x = np.arange(12)          # 0..11
A = x.reshape((3, 4))      # 3x4 matrix

A.T                         # transpose: 4x3
```

Key idea: reshape changes the shape, not the values.

DOT PRODUCTS AND MATRIX PRODUCTS

Two common operations:

Dot product of vectors (or general inner products):

```
x = np.array([1, 2, 3])  
y = np.array([4, 5, 6])
```

```
np.dot(x, y)      # 32
```

Matrix multiplication:

```
A = np.array([[1, 2],  
              [3, 4]])  
B = np.array([[5, 6],  
              [7, 8]])
```

```
A @ B
```

SOLVING LINEAR SYSTEMS

A core scientific task is solving $Ax = b$.

```
import numpy as np

A = np.array([[3.0, 1.0],
              [1.0, 2.0]])
b = np.array([9.0, 8.0])

x = np.linalg.solve(A, b)
```

Key idea: this avoids explicitly computing A^{-1} and uses stable numerical methods internally.

RANDOM NUMBERS (AND WHY SEEDING MATTERS)

Random simulation is common in scientific computing.

```
rng = np.random.default_rng(0)    # seed for reproducibility  
  
x = rng.normal(0, 1, size=1000)  # 1000 samples from  $N(0,1)$ 
```

If you do not set a seed, results will differ each run. For science, reproducibility matters: you want to be able to regenerate the same outputs.

NUMPY SUMMARY

NumPy is the foundation of scientific Python because it provides:

- ▶ an efficient numerical array type (**ndarray**),
- ▶ vectorized operations that avoid Python loops,
- ▶ fast linear algebra and reductions,
- ▶ clean indexing, slicing, and masking tools.

Big picture: NumPy lets you write mathematical code that is both readable and efficient, which is exactly what scientific programming requires.

WHAT WE WILL PRACTICE NEXT

In the next segment, we will practice NumPy by:

- ▶ creating arrays and checking shapes,
- ▶ writing vectorized transformations and filters,
- ▶ doing simple linear algebra operations,
- ▶ comparing a Python-loop approach with a NumPy-vectorized approach.

Goal: you should become comfortable treating arrays as mathematical objects, not lists.